

Q1

Given:

- Relation size = 2000 blocks
- Sorted on id
- Seek time = 10 ms
- Block transfer time = 0.1 ms
- Blocks with id ≥ 1801111042 = 500
- Height of primary index on id = 3

Query:

SELECT * FROM student WHERE id > 1801111042;

Cost using primary index:

1. Search cost (using index): Need to traverse B+ tree of height 3 $\rightarrow$ cost = $3 \times (10 + 0.1) = 30.3$ ms
2. Fetch matching blocks: 500 blocks
 $\rightarrow$ 1 seek + $(500 \times 0.1 \text{ ms transfer}) = 10 + 50 = 60$ ms

✓ Total cost = $30.3 + 60 = \approx 90.3$ ms

Q2

Query:

SELECT * FROM student WHERE id < 1801111042;

Since 500 blocks are $\geq$ the id, that means 1500 blocks are < 1801111042.

Two options:

- Using index: Need to scan first 1500 blocks sequentially.
- Without index: Full table scan also 2000 blocks.

So we choose cheaper:

Index search (height 3) + sequential scan of 1500 blocks

= $(3 \times 10.1) + (1 \text{ seek} + 1500 \times 0.1 \text{ ms})$

= $30.3 + 10 + 150 = \approx 190.3$ ms

Q3

Query:

SELECT * FROM student WHERE city = 'Dhaka';

Given:

- B+ tree on city (height = 3)
- 200 tuples (each in separate block)

Steps:

1. Search city='Dhaka' $\rightarrow 3 \times (10 + 0.1) = 30.3$ ms
2. Access 200 blocks (each tuple separate):
 $200 \times (10 + 0.1) = 200 \times 10.1 = 2020$ ms

✓ Total $\approx 30.3 + 2020 = \approx 2050$ ms

Q4

Query:

SELECT *

FROM student

WHERE CGPA=3 AND City='Dhaka' AND year-admit=2020;

Indices:

- CGPA $\rightarrow$ 3-level (100 tuples)
- City $\rightarrow$ 3-level (200 tuples)
- year-admit $\rightarrow$ 2-level (1000 tuples)

- Each tuple in a separate block.

Algorithms:

1. Single index (use one, check others)
2. Intersection of record pointers (ANDing of indices)
3. Composite index (if exists — not here)

We'll calculate for (1) and (2).

(a) Using single index (City):

Search cost = $3 \times 10.1 = 30.3$ ms

Fetch = $200 \times 10.1 = 2020$ ms

Then filter in memory by CGPA = 3 and year = 2020 (negligible).

➡ ≈ 2050 ms

(b) Intersection of indices (ANDing):

Search 3 indices $\rightarrow 3 \times (10.1 + 10.1 + 10.1) \approx 30.3 + 30.3 + 30.3 = \approx 90$ ms

Smallest list = CGPA = 3 (100 tuples)

For each of 100 tuples, check matches $\rightarrow 100 \times 10$ ms = 1000 ms

✓ ≈ 1080 ms

Query optimizer picks (b) since it's cheaper.

Q5

Schemas:

Student(Sid, name, street, city, mobile, email, CGPA, age)

Takes(course-id, Sid, semester, year, GP)

Course(course-id, title, credit-hour)

Indices:

Primary on all PKs. Secondary on name, street, city, age, title.

(i)

SELECT * FROM student WHERE sid > 10001;

→ Use primary index on sid (sorted).

→ Find first sid > 10001 using index → sequentially fetch rest of data blocks.

→ Similar to Q1 pattern.

(ii)

SELECT * FROM course WHERE title = 'Database';

→ Use secondary index on title (B+ tree).

→ Traverse index (height maybe 3) + fetch record blocks.

(iii)

SELECT *

FROM takes

WHERE sid = 10005 AND course_id = 'CSE311'

AND semester = 'Spring' AND year = 2022;

→ Use primary index on (course_id, sid, semester, year) (assuming composite).

→ Only one matching record → cost = height of index $\times (10 + 0.1) \approx$ few ms.

Q6

Relation sorted on NID.

Indices: id (height 3) and NID.

Query:

SELECT * FROM student WHERE id = 1801111042;

→ Use id index (not NID)

→ Search cost = $3 \times (10 + 0.1) = 30.3$ ms

→ Fetch the record block ($1 \times (10 + 0.1) = 10.1$ ms)

✓ Total ≈ 40.4 ms

Q7

Student(id, name, CGPA, street, city, year-admit)

Takes(id, course-id, semester, year)

Relation Records Blocks

student 1000 50

takes 5000 10

(a) Nested Loop Join

Algorithm:

for each record in student:

for each record in takes:

if student.id = takes.id:

output join tuple

- Cost = $(bR + (bR \times bS))$

- $bR = 50, bS = 10$

- $= 50 + (50 \times 10) = 550$ block accesses

Best case: Matching tuples found early → same cost pattern

Worst case: All blocks read → 550

(b) Block Nested Loop Join

Algorithm:

for each block of student:

for each block of takes:

compare and output matches

Cost = $bR + (bR \times bS)$ but with block-wise caching benefit.

If buffer can hold all takes (say all 10 blocks fit):

→ Best case = $bR + bS = 50 + 10 = 60$ blocks

→ Worst case = $50 + (50 \times 10) = 550$ blocks

Q9

Schemas same as Q5.

(a)

SELECT sid, name

FROM student INNER JOIN takes ON student.sid = takes.sid

WHERE city = 'Dhaka' AND year = 2023;

Relational Algebra:

$\pi_{sid, name} (\sigma_{city='Dhaka' \wedge year=2023} (Student \bowtie Takes))$

Execution Tree:

$\pi_{sid, name}$

|

$\sigma_{city='Dhaka' \wedge year=2023}$

|

(b)

SELECT sid, name

FROM (student ⋈ takes) ⋈ course

WHERE city='Dhaka' AND year=2023 AND credit-hour=3;

Relational Algebra:

 $\pi_{sid, name} (\sigma_{city='Dhaka' \wedge year=2023 \wedge credit-hour=3}$ $((Student \bowtie Takes) \bowtie Course))$

Execution Tree:

 $\pi_{sid, name}$

|

 $\sigma_{city='Dhaka' \wedge year=2023 \wedge credit-hour=3}$

|

 $(Student \bowtie Takes) \bowtie Course$



Q10

Explain query processing methods

(i) Materialized method

- Each intermediate result (e.g., Student ⋈ Takes) is stored temporarily.
- Next operations use this stored table.
- Slower, needs more I/O and storage.

(ii) Pipeline method

- Intermediate results not stored; directly passed to next operation in memory.
- Faster, less I/O, but more CPU load.

Q1 — RAID level 5 & RAID 1 examples**Part A — “4 disks of 2TB each; student relation blocks B1..B9; show storage in RAID-5”****Background / assumptions**

- RAID-5 with 4 disks: each stripe contains 3 data blocks + 1 parity block (parity rotates).
- Disks: D1, D2, D3, D4.
- We place blocks in stripe order, rotating parity left-to-right each stripe (common convention).

Stripe layout (3 data + 1 parity per stripe).

We have 9 data blocks; each stripe holds 3 data blocks → need 3 stripes.

Stripe numbering and layout (parity shown as P#):

Stripe Disk D1 Disk D2 Disk D3 Disk D4

0	B1	B2	B3	P0
1	B4	B5	P1	B6
2	B7	P2	B8	B9

Explanation of rotation:

- Stripe 0 parity stored on D4 (P0).
- Stripe 1 parity stored on D3 (P1).
- Stripe 2 parity stored on D2 (P2).

So the final placement:

- D1: B1, B4, B7
- D2: B2, B5, P2
- D3: B3, P1, B8
- D4: P0, B6, B9

This shows parity distribution and data blocks across 4 disks.

Part B — “4 TB disks; need 20 TB effective storage using RAID-1; total number of disks? Then store B1..B10.”**RAID-1 (mirroring) basics**

- RAID-1 mirrors each disk; effective capacity = $(\text{total disks} / 2) \times \text{disk_size}$ (assuming pairs).
- With 4 TB disks, effective per mirrored pair = 4 TB.

Compute number of disks

- Required effective = 20 TB.
- #pairs needed = $20 \text{ TB} \div 4 \text{ TB/pair} = 5 \text{ pairs}$.
- Total disks = $5 \text{ pairs} \times 2 \text{ disks/pair} = \mathbf{10 \text{ disks}}$.

Storage of B1..B10 on RAID-1 (pair mapping)

We'll number disks as mirrored pairs: (D1,D2), (D3,D4), (D5,D6), (D7,D8), (D9,D10).

Place each block on both members of a pair:

Block Replica on Pair

B1 (D1, D2)
B2 (D1, D2)
B3 (D3, D4)
B4 (D3, D4)
B5 (D5, D6)
B6 (D5, D6)
B7 (D7, D8)
B8 (D7, D8)
B9 (D9, D10)
B10 (D9, D10)

(Distribution above is an example — in practice a filesystem/RAID controller stripes pages/blocks across pairs for load balancing; the important point is each block is mirrored.)

Q2 — RAID-5 capacity for 20 TB using 4 TB disks (and map B1..B10)

Problem: ecommerce system requires 20 TB effective with RAID-5; disk size = 4 TB. Find number of disks and show placement of blocks B1..B10.

RAID-5 capacity rule

- Effective capacity = $(N - 1) \times \text{disk_size}$ (one-disk worth of parity each stripe).
- Need $(N - 1) \times 4 \text{ TB} \geq 20 \text{ TB} \rightarrow N - 1 \geq 5 \rightarrow N \geq 6$.

Minimum number of disks = 6.

Check: $(6 - 1) \times 4 \text{ TB} = 20 \text{ TB} \rightarrow \text{exact}$.

Map B1..B10 into RAID-5 with N = 6 (disks D1..D6).

- Now each stripe contains 5 data blocks + 1 parity block.
- We'll place blocks sequentially; parity rotates.

Stripe 0 (blocks B1..B5) + parity P0:

Disk Content

D1 B1
D2 B2
D3 B3
D4 B4
D5 B5
D6 P0

Stripe 1 (next blocks B6..B10) + parity P1:

Disk Content

D1 B6

D2 B7

D3 B8

D4 B9

D5 B10

D6 P1 (if parity rotates we could put parity on a different disk; this table places parity on D6 again for simplicity, but a rotation convention would move P1 to D5, etc.)

(If strictly rotating parity: Stripe1 parity would be on D5, and data placed on remaining disks; either approach is acceptable as long as parity rotates.)

This places the 10 blocks across the six disks with parity blocks included.

Q3 — Sparse index on c-id and index on name (given 12 rows)

Given relation (12 rows):

c-id | Name | City

C-1 | Fatema | Dhaka

C-2 | Sharif | Comilla

C-3 | Zahid | Khulna

C-4 | Abid | Dhaka

C-5 | Fatema | Comilla

C-6 | Sharif | Khulna

C-7 | Zahid | Dhaka

C-8 | Abid | Comilla

C-9 | Sharif | Khulna

C-10 | Zahid | Comilla

C-11 | Abid | Khulna

C-12 | Fatema | Dhaka

(a) Construct a sparse index on c-id

Assumption: A block can hold 3 records (common simple assumption). So blocks are:

- Block B1: C-1, C-2, C-3
- Block B2: C-4, C-5, C-6
- Block B3: C-7, C-8, C-9
- Block B4: C-10, C-11, C-12

Sparse index (store first key of each block):

Index Entry Pointer to Block

C-1 → B1

C-4 → B2

C-7 → B3

C-10 → B4

To find any c-id: find the largest index entry $\leq$ desired key, then scan that block.

(b) Construct index on Name

Because Name has duplicates and user likely wants exact matches, a **dense secondary index** is typical for Name (one index entry per record), or an index with buckets (inverted list) mapping each Name to a list of record pointers.

Dense index example (sorted by Name, then c-id pointers):

Name List of c-id pointers

Abid → C-4, C-8, C-11

Fatema → C-1, C-5, C-12

Sharif → C-2, C-6, C-9

Zahid → C-3, C-7, C-10

This is effectively a secondary index mapping each distinct name to the list of record pointers (bucket).

(c) Create a sparse index on c-id where C-5 and C-6 will not have index entry. Find c-id = C-6 using the index.

Interpretation / assumption: We create sparse index entries for some blocks but deliberately omit entries for blocks that contain C-5 and C-6. Using the earlier block grouping:

- B2 contains C-4, C-5, C-6. We will *not* include an index entry for the block B2 (so no C-4 entry either in this version).

Modified sparse index (skipping B2):

Index Entry Pointer

C-1 → B1

C-7 → B3

C-10 → B4

Finding C-6 using this sparse index

1. Search index for largest entry $\leq$ C-6: entries $\leq$ C-6 are C-1. Next entry C-7 $>$ C-6.
2. So we follow pointer $\rightarrow$ B1 (but B1 contains C-1..C-3), not the block that has C-6.
3. Because the sparse index omitted B2, the index leads us to the previous block; we must then scan forward block(s) until we find C-6. So:
 - Scan B1 $\rightarrow$ not found.
 - Read next physical block B2 $\rightarrow$ find C-6.

Conclusion: Even if B2 has no index entry, the sparse index still locates the correct block by choosing the preceding entry and then scanning forwards. But omission increases read cost (one extra block read).

Q4 — Compare RAID 1 and RAID 5 (with examples)

RAID-1 (Mirroring)

- **Structure:** Each disk has an exact copy (mirror) on another disk (usually mirrored in pairs).
- **Storage efficiency:** 50% (N disks $\rightarrow N/2$ effective capacity).
- **Fault tolerance:** Can survive failure of one disk per mirrored pair (multiple failures possible if different pairs fail).
- **Performance:** Read performance improved (reads can be load-balanced), write performance similar to single disk (writes must go to both mirrors).
- **Best use:** Small arrays requiring high read performance and simple rebuilds; frequent updates are fine; simple recovery.
- **Example:** Two 4TB drives in RAID-1 $\rightarrow$ effective 4TB, mirrored copy on second disk.

RAID-5 (Striping with distributed parity)

- **Structure:** Data striped across N disks with parity distributed across disks. Effective capacity = $(N-1) \times \text{disk_size}$.
- **Storage efficiency:** $(N-1)/N$ (better as N grows).
- **Fault tolerance:** Can survive one disk failure (rebuild costs I/O).
- **Performance:** Good read performance (striping). Write penalty because parity must be updated (read-modify-write or full stripe).
- **Best use:** Balanced read performance with more storage efficiency; common for large capacity arrays with fewer writes or where write penalty is acceptable.
- **Example:** Six 4TB disks in RAID-5 $\rightarrow$ effective 20 TB (5×4 TB) with one-disk parity.

Quick comparison table

Feature	RAID-1	RAID-5
Redundancy	Mirror (full copy)	Distributed parity (1 disk worth)
Capacity eff.	50%	$(N-1)/N$ (better for larger N)
Read perf.	Good (reads from mirrors)	Good (striping)
Write perf.	Similar to single disk (dup)	Worse (parity overhead)
Fault tolerance	Can lose one disk in each pair	Can lose one arbitrary disk
Best for	Small arrays, frequent updates	Large arrays, read-heavy workloads

Q5 — Explain worst-case and best-case seek time and block transfer time

Definitions

- **Seek time:** time disk head spends moving to the correct track. Depends on current head position and target track.
 - **Best-case seek:** 0 (or near-zero) if head already at target track or for perfectly sequential access (no seek).
 - **Worst-case seek:** full stroke (max distance) — device-specific (e.g., 8 ms–20 ms typical for HDDs); can be modeled as `seek_max`.
- **Block transfer time:** time to actually transfer the block data once head is in place and rotational latency has been accounted for. Depends on block size and disk transfer rate.
 - **Best-case transfer:** transfer time for contiguous blocks with no intervening seeks; for a single block it's `block_size / transfer_rate`.
 - **Worst-case transfer:** transfer time itself doesn't vary much, but effective time per block increases if each block requires a separate seek/rotation.

Access time model (simplified):

- Access time $\approx$ Seek time + Rotational latency + Transfer time
 - Rotational latency best case $\approx$ 0 (if sector under head) ; worst case = one full rotation ($\frac{1}{2}$ rotation average often used).
 - Transfer time depends on block bytes and disk throughput.

Examples

- **Best case (sequential scan):** head already on track, sectors aligned $\rightarrow$ each block read costs roughly only transfer time (seek $\approx$ 0, rotation $\approx$ small) $\rightarrow$ low per-block time.
- **Worst case (random scattered reads):** each block on different track and sector $\rightarrow$ for each block you pay full seek + average rotational latency + transfer $\rightarrow$ very high per-block time.

Practical numbers (example from earlier problems)

- Seek time = 10 ms (given) is the typical **seek** cost per random access.
- Block transfer = 0.1 ms is the time to transfer a block once head positioned.
- So random single-block access $\approx 10 + 0.1$ ms ≈ 10.1 ms (ignoring rotation) — best and worst contexts change how often you pay 10 ms.

Q6 — Design for 10 TB reliable storage with 2 TB disks; frequent updates required

Requirements

- Disk size = 2 TB.
- Need 10 TB effective.
- System must be reliable and support **frequent updates**.

Recommendation: Use **RAID-10 (1+0)** — mirrored pairs striped across pairs.

- **Why RAID-10?** It combines the write performance and low rebuild complexity of mirroring with the striping/performance of RAID-0. For frequent updates, RAID-10 handles writes better than parity RAID (like RAID-5/6) because no parity updates needed.
- **Capacity math:** RAID-10 effective capacity = $(N/2) \times \text{disk_size}$ (same as mirrored arrays).

- Need: $(N/2) \times 2 \text{ TB} \geq 10 \text{ TB} \rightarrow N/2 \geq 5 \rightarrow N \geq 10$.
- **So use 10 disks (5 mirrored pairs)** of 2 TB each $\rightarrow$ effective 10 TB.

Layout example (5 pairs):

- Pairs: (D1,D2), (D3,D4), (D5,D6), (D7,D8), (D9,D10).
- Data is striped across the 5 pairs; each stripe written to one disk of each pair and mirrored to its pair.

Advantages

- High write throughput and low write penalty (no parity).
- Good fault tolerance: can survive multiple disk failures as long as both disks in a mirrored pair do not fail simultaneously.
- Faster rebuilds because rebuilding requires copying from a single mirror.

Q7 — Construct an index on dept_name for the following relation

(The file did not include the exact relation attributes for Q7; typical approach:)

Assumptions

- Relation: Dept(dept_id, dept_name, manager, location, ...)
- We want an index on dept_name. dept_name may be non-unique.

Two common index choices

1. **B+ tree (ordered index)** — good for range queries, prefix search, ordered retrieval.
2. **Hash index** — good for equality lookup only (fast exact matches), not for ranges.

Suggested index: B+ tree on dept_name (dense or secondary index).

- **Structure:** Leaf nodes contain (dept_name, pointer/list_of_recids). For duplicate dept_names, store a list (bucket) of record pointers.
- **Example leaf entries (sample):**

dept_name Record pointers

CSE	$\rightarrow$ DeptIDs {10, 37, 98}
EEE	$\rightarrow$ DeptIDs {12}
Math	$\rightarrow$ DeptIDs {21, 43}
Physics	$\rightarrow$ DeptIDs {8}

How it helps

- `SELECT * FROM Dept WHERE dept_name = 'CSE'` $\rightarrow$ single B+ tree search $\rightarrow$ get list of records quickly.
- Range queries like `dept_name >= 'C' AND dept_name < 'M'` are efficient with B+ tree.